惠聚项目-技术概要设计方案

一、引言

1.1. 编写目的

本文档旨在阐述"惠聚项目"的系统级设计方案，为后续的详细设计、开发、测试及运维工作提供统一的架构蓝图和核心技术指引。通过本文档，确保项目各方对系统整体设计有清晰、一致的理解，明确各系统的职责边界、核心业务流程、技术实现方案及非功能性设计要求，从而指导研发团队高效、高质量地完成系统建设。

1.2. 项目背景

为提升存量用户的价值转化并探索新的业务增长点，科技平台拟推出"拒件导流"模式。当部分用户在科技平台申请用信被拒后，系统将其引导至合作方的权益购买页面。用户购买权益后，再由"云卡"作为中转枢纽，引导用户回到科技平台完成后续的试算、签约和放款流程。此模式涉及科技平台、艾博生、云卡三方系统的深度协同，需要构建一套稳定、高效、安全的跨机构系统架构来支撑业务闭环。

1.3. 名词解释

术语	英文/缩写	解释
科技平台	KJ	本项目的核心资金方和资产方，负责风控、放款和还款。
艾博生	ABS	业务合作方，作为甲方售卖权益产品，是用户流量的第一承接方。
云卡	YK	业务中转枢纽，负责连接艾博生和科技平台，提供试算、撮合签约等服务。
拒件导流	-	科技平台将不符合其授信标准的用户，推荐给艾博生的业务模式。
权益先享后付	-	用户在艾博生处购买权益，若首次代扣失败，可在科技平台放款成功后，由艾博生进行二次兜底代扣的业务模式。

二、设计目标与原则

2.1. 设计目标

目标分类	目标描述	衡量指标

性能	核心交易接口（如下单、支付、放款）99%响应时间在500ms以内， TPS（每秒事务处理量）峰值不低于500。	P99 RT < 500ms; Peak TPS > 500
可扩展性	系统支持水平扩展，可通过增加应用节点线性提升处理能力，以应对未来业务量增长。数据库支持分库分表，单表数据量控制在2000万以内。	应用节点可无状态水平扩展; 数据库可平滑扩容
可用性	核心业务链路可用性达到99.95%，非核心业务可用性达到99.9%。关键服务采用集群化部署，避免单点故障。	核心SLA >= 99.95%; 关键服务无单点
安全性	杜绝资金损失事件。敏感数据（如身份证、手机号、银行卡）必须加密存储，系统间通信必须使用HTTPS/mTLS加密。	资损事件 = 0; 敏感数据加密存储; 通信链路全加密

2.2. 设计原则

高内聚，低耦合是本系统架构设计的首要原则。各系统、各模块职责单一、边界清晰，模块内部紧密协作，模块之间通过定义良好的接口进行交互。这意味着科技平台、云卡、艾博生三方系统各自独立演进，互不干扰。

面向未来，分步实施。架构设计应具备一定的前瞻性，能够支持未来2-3年的业务发展。但实施上可以分阶段进行，优先满足当前核心业务需求，避免过度设计带来的资源浪费。

服务化与微服务。对于科技平台等复杂系统，采用微服务架构，将核心业务能力（如风控、订单、支付）沉淀为独立的服务，便于独立开发、部署和扩展。云卡和艾博生作为相对轻量的系统，可在初期采用单体架构，后续根据业务发展再进行服务拆分。

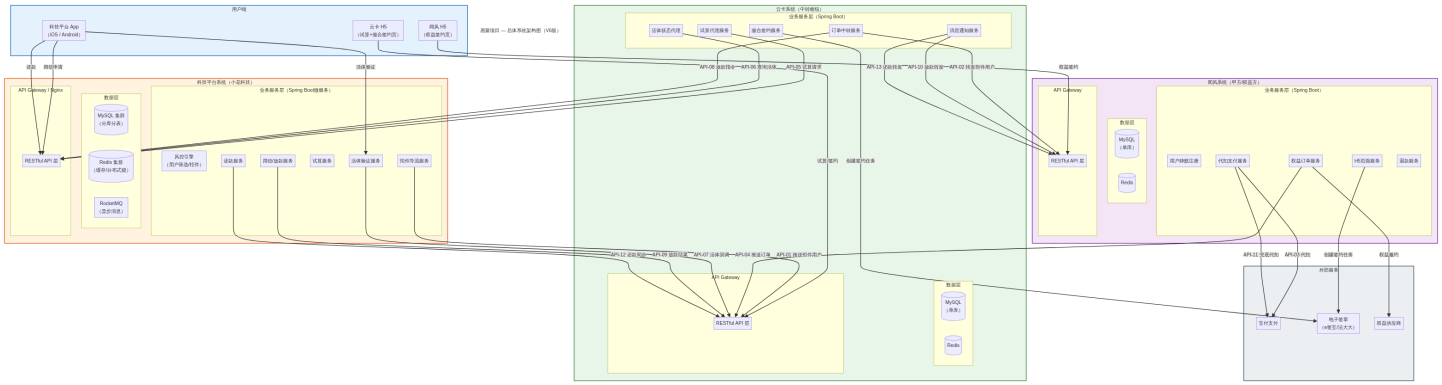
数据驱动。建立完善的监控和日志体系，所有决策和优化都应基于数据分析，而非主观臆断。从业务指标到技术指标，都需要有清晰的数据采集和展示能力。

安全第一。在设计的每一个环节（应用、网络、数据、运维）都必须充分考虑安全性，构建纵深防御体系。尤其是在涉及资金流转和用户敏感信息的场景中，安全是不可妥协的底线。

三、总体架构设计

3.1. 系统全景架构

系统整体由用户端、科技平台系统、云卡系统、艾博生系统及外部依赖服务五大部分组成，形成一个完整的业务与数据闭环。下图展示了各系统之间的关系和核心交互链路。



用户通过科技平台App发起借款，后续会跳转至艾博生和云卡的H5页面完成权益购买和撮合签约。科技平台、云卡、艾博生三方系统各司其职，通过API进行串联。其中，云卡作为中转枢纽，起到了业务解耦和流程控制的关键作用，使得科技平台和艾博生之间无需直接交互，降低了系统耦合度。宝付支付、电子签章、权益供应商是支撑业务完成的重要外部依赖。

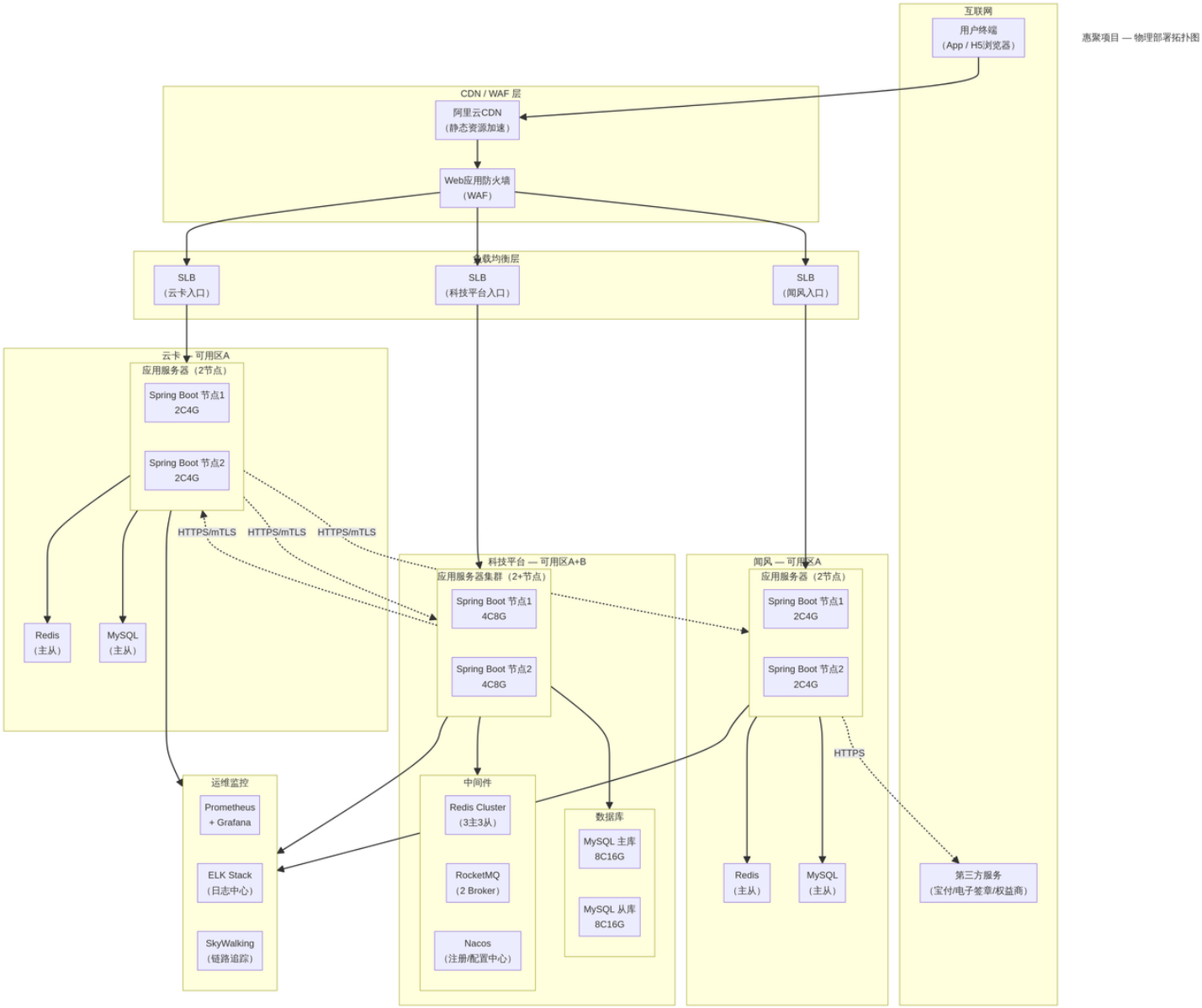
3.2. 技术栈选型

为保证技术统一性、研发效率和生态成熟度，三方核心后端服务均采用以Java为主的技术栈。以下是各技术领域的选型及理由：

技术领域	选用技术	选型理由
核心框架	Spring Boot 2.7.x + Spring Cloud 2021	成熟稳定，生态完善，是Java微服务开发的事实标准，人才储备丰富。
数据库	MySQL 8.0 + MyBatis-Plus	开源、稳定、社区活跃。MyBatis-Plus提供强大ORM能力，简化开发。
缓存	Redis 6.x + Redisson	高性能KV缓存，支持丰富数据结构。Redisson提供强大的分布式锁和集合实现。
消息队列	RocketMQ 4.x（待评估）	阿里开源，金融级可靠性，支持事务消息和顺序消息，满足支付、订单等场景需求。
服务治理	Nacos 2.x（暂时不用）	集服务发现、配置管理于一体，与Spring Cloud生态无缝集成。
网关	Spring Cloud Gateway	Spring官方网关，基于WebFlux，性能高，易于扩展，可实现鉴权、限流、路由等功能。
任务调度	XXL-JOB	成熟的分布式任务调度平台，提供可视化管理、分片广播、故障转移等能力。
监控告警	Prometheus + Grafana + Alertmanager	开源监控领域的黄金组合，提供强大的数据采集、展示和告警能力。

日志审计	ELK Stack (Elasticsearch, Logstash, Kibana)	业界标准的日志解决方案，提供从采集、存储到查询分析的全套能力。
链路追踪	SkyWalking（暂时不用）	国产开源APM，对Java应用无侵入式探针，提供拓扑、追踪、指标等丰富功能。

3.3. 物理部署拓扑



3.4. 逻辑分层架构

1）接入层 (Presentation Layer) 负责处理HTTP请求，进行协议转换、参数校验、安全认证等，是系统对外的直接入口。

2）应用层 (Application Layer) 负责编排领域服务，协调多个领域对象完成一个完整的业务场景，处理事务、发布事件等。

3）领域层 (Domain Layer) 包含所有业务逻辑的核心，如领域对象（Entity, VO）、领域服务（Domain Service）和仓储接口（Repository），此层不依赖任何外部实现。

4）基础设施层 (Infrastructure Layer) 为其他各层提供通用的技术能力支持，如数据库访问、缓存操作、消息发送、第三方接口调用等，是领域层仓储接口的具体实现。

3.5 部署架构

线下环境（1个实例）

· 用途：开发、测试、演示。

· 配置建议：4核8G（或8核16G），单机部署所有组件（MySQL、Redis、Nacos、Seata、应用服务）。

· 优势：成本极低，环境统一，便于快速迭代。

线上环境（6个实例，双机房部署）

· 云卡 (nexusfin-bridge)：2个实例（跨机房部署），保证高可用。

· 艾博生 (nexusfin-equity)：2个实例（跨机房部署），保证高可用。

· 数据库 (MySQL)：2个实例（主从架构，跨机房部署），保证数据安全。

· 合计：6个线上实例。

建议：使用阿里云，推荐直接购买 云产品 (RDS, Redis, MQ)，虽然成本略高，但免运维、高可用，且不占用这 6 个实例的资源。

四、核心业务流程

4.1. 业务流程概述

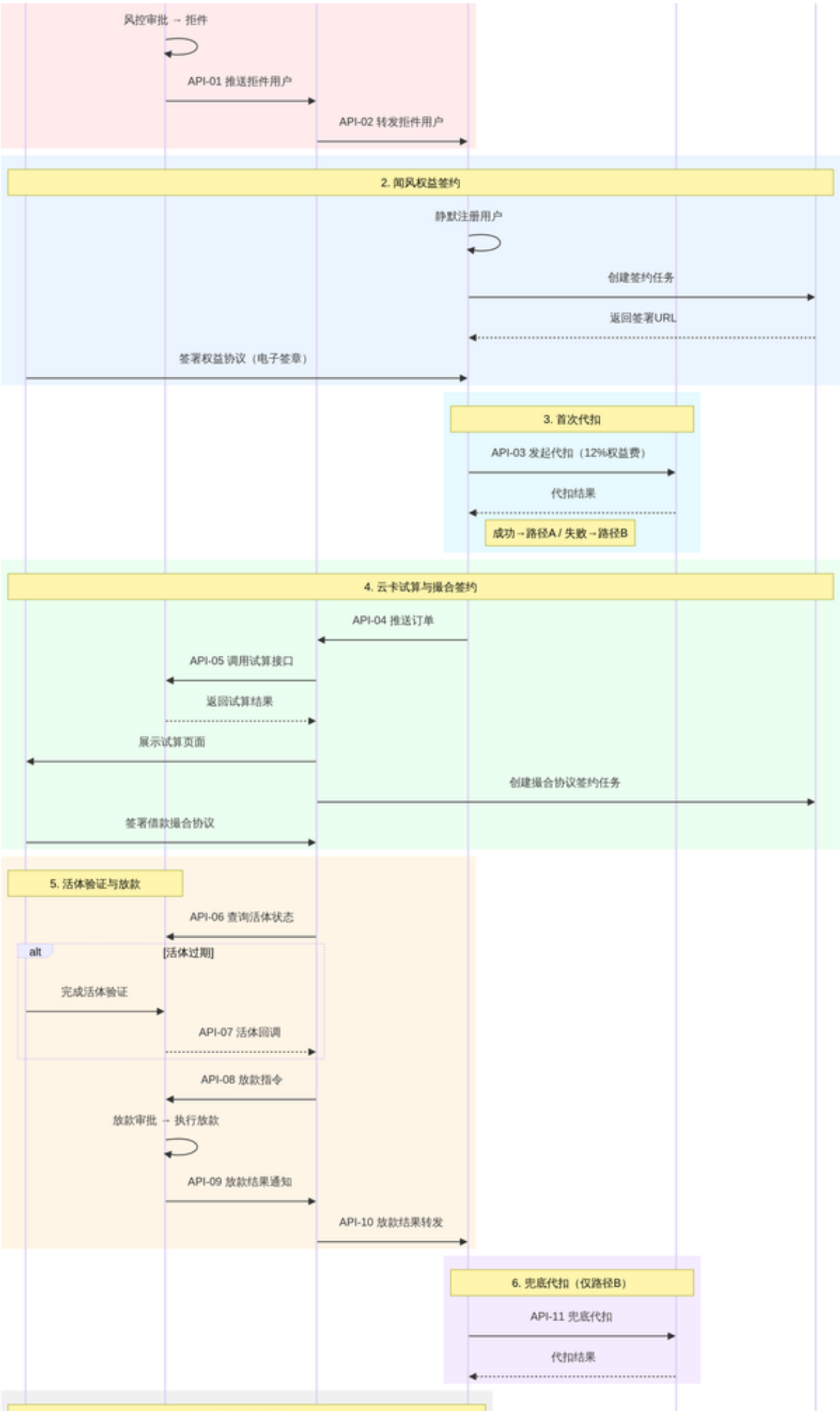
惠聚项目V6版的核心是"拒件导流"和"权益先享后付"两个机制，整个流程涉及七个主要阶段，并存在两条核心路径：

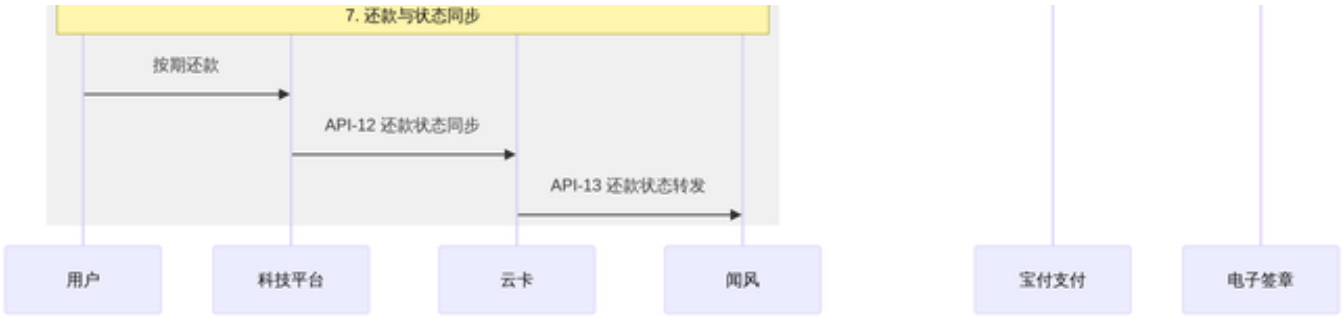
路径A（首扣成功路径）：用户在艾博生处首次代扣权益费成功，直接进入后续的云卡试算、签约和放款流程。

路径B（先享后付路径）：用户首次代扣失败，进入"先享后付"模式，先完成借款流程，待科技平台放款成功后，由艾博生再次发起兜底代扣。

4.2. 核心借款时序图

下图详细描述了从用户申请到最终还款的全链路处理时序，并标注了各系统间的API交互。


风控审批 → 拒件
API-01 推送拒件用户
API-02 转发拒件用户
2. 闻风权益签约
静默注册用户
创建签约任务
返回签署URL
签署权益协议（电子签章）
3. 首次代扣
API-03 发起代扣（12%权益费）
代扣结果
成功→路径A / 失败→路径B
4. 云卡试算与撮合签约
API-04 推送订单
API-05 调用试算接口
返回试算结果
展示试算页面
创建撮合协议签约任务
签署借款撮合协议
5. 活体验证与放款
API-06 查询活体状态
alt
[活体过期]
完成活体验证
API-07 活体回调
API-08 放款指令
放款审批 → 执行放款
API-09 放款结果通知
API-10 放款结果转发
6. 兜底代扣（仅路径B）
API-11 兜底代扣
代扣结果



阶段一：拒件导流。科技平台风控拒绝用户后，通过API-01将用户信息推送给云卡，云卡再通过API-02转发给艾博生。

阶段二：艾博生权益签约。艾博生为用户静默注册，调用电子签章服务创建签约任务，引导用户在H5页面签署《权益服务协议》和《权益先享后付协议》。

阶段三：首次代扣。艾博生通过宝付支付（API-03）尝试扣取12%的权益费。代扣成功进入路径A，失败进入路径B。

阶段四：云卡试算与撮合签约。艾博生将订单通过API-04推送给云卡。云卡调用科技平台试算接口（API-05），获取用户可贷额度、利率、期数等信息，并在H5试算页面展示给用户。随后，云卡调用电子签章服务创建签约任务，引导用户签署《借款撮合协议》。

阶段五：活体验证与放款。云卡通过API-06查询用户在科技平台的活体状态。若活体已过期，则通过DeepLink调起科技平台App，引导用户完成活体验证，科技平台通过API-07回调通知结果。验证通过后，云卡通过API-08向科技平台发起放款指令，科技平台完成审批和放款后，通过API-09通知云卡，云卡再通过API-10转发给艾博生。

阶段六：兜底代扣（仅路径B）。艾博生在收到放款成功通知后，通过宝付支付（API-11）再次发起代扣。

阶段七：还款与状态同步。用户在科技平台按期还款后，科技平台通过API-12将还款状态同步给云卡，云卡再通过API-13转发给艾博生。

五、模块详细功能分配

为实现业务目标，三方系统需各自建设或改造一系列功能模块，职责分配如下：

5.1. 科技平台 (KJ)

模块名称	核心功能描述
拒件导流模块	根据风控结果，筛选出符合导流标准的拒件用户。调用云卡接口（API-01），将用户脱敏信息推送至下游。
用户与活体模块	管理核心用户信息及KYC状态。提供活体状态查询接口（API-06）。提供App端活体验证能力，并通过回调（API-07）通知云卡结果。
试算服务模块	

	接收云卡的试算请求（API-05）。根据用户ID和申请金额，结合内部风控和定价模型，计算可贷额度、利率、期数等，返回详细的试算结果给云卡。
订单与放款模块	接收云卡的放款指令（API-08）。对放款请求进行最终校验（权益订单状态、撮合协议、活体状态等），执行放款操作并记录结果。调用云卡接口（API-09），异步通知放款结果。
还款与账务模块	处理用户的按期还款，生成还款计划。调用云卡接口（API-12），同步还款状态信息。

5.2. 云卡 (YK)

模块名称	核心功能描述
订单中转模块	接收科技平台的拒件用户（API-01），并转发给艾博生（API-02）。接收艾博生的权益订单（API-04），并存储关键信息。
试算与签约模块	提供H5试算页面。调用科技平台试算接口（API-05）获取数据并展示。调用电子签章服务，引导用户签署《借款撮合协议》。
消息通知模块	接收科技平台的放款结果（API-09），并转发给艾博生（API-10）。接收科技平台的还款状态（API-12），并转发给艾博生（API-13）。
活体代理模块	调用科技平台接口（API-06）查询用户活体状态。根据结果决定是否通过DeepLink调起科技平台App进行活体验证。接收科技平台的活体验证回调（API-07）。

5.3. 艾博生 (ABS)

模块名称	核心功能描述
用户与H5模块	接收云卡转发的用户信息（API-02），为用户进行静默注册。提供权益购买和签约的H5页面。
权益订单模块	用户签约后，创建权益订单。调用云卡接口（API-04），将已支付或待兜底的订单信息推送至下游。
签约模块	调用电子签章服务，创建签约任务，引导用户签署《权益服务协议》和《权益先享后付协议》。
支付与退款模块	调用宝付支付，发起首次代扣（API-03）和兜底代扣（API-11）。处理支付回调，更新订单状态。提供线下退款支持流程。

六、接口设计方案

系统间的交互主要通过RESTful API进行。内部微服务之间可采用性能更优的RPC（如Dubbo或gRPC），但为简化三方对接复杂度，跨系统接口统一规范为基于HTTP的RESTful风格。

6.1. 接口设计原则

RESTful风格。使用标准的HTTP方法（GET, POST, PUT）表达操作意图，URL使用名词复数形式。所有请求和响应体均使用application/json格式。

统一响应结构。所有API响应体都封装在统一的结构中，包含状态码（code，0表示成功）、消息（message）和业务数据（data）。

安全性。传输层面，所有接口必须使用HTTPS协议。认证层面，系统间调用采用基于AppID + AppSecret的签名机制，请求头中包含AppID、时间戳Timestamp、随机数Nonce和签名Signature，签名由请求参数、Timestamp、Nonce和AppSecret通过HMAC-SHA256算法生成。此外，在网关层配置调用方服务器的IP白名单，作为第一层防护。

幂等性。所有POST和PUT请求，凡是涉及状态变更或资源创建的，都必须支持幂等。调用方需在请求中传递唯一的请求流水号（如request_id），服务端通过该流水号进行幂等判断，防止因网络重试导致的重复操作。

6.2. 核心接口清单 (13个)

编号	接口名称	调用方向	HTTP方法	核心功能描述
API-01	推送拒件用户	KJ → YK	POST	科技平台将拒件用户信息推送给云卡。
API-02	转发拒件用户	YK → ABS	POST	云卡将用户信息转发给艾博生，用于静默注册。
API-03	发起代扣	ABS → 宝付	POST	艾博生调用宝付支付，扣取首次权益费。
API-04	推送权益订单	ABS → YK	POST	艾博生将创建的权益订单信息推送给云卡。
API-05	借款试算	YK → KJ	POST	云卡请求科技平台对用户进行借款试算。
API-06	查询活体状态	YK → KJ	GET	云卡查询用户在科技平台的活体状态是否有效。
API-07	活体验证回调	KJ → YK	POST	用户在科技平台App完成活体验证后，结果回调给云卡。
API-08	发起放款	YK → KJ	POST	云卡校验通过后，向科技平台发起最终的放款指令。
API-09		KJ → YK	POST	科技平台异步通知云卡放款的最终结果。

	放款结果通知			
API-10	转发放款结果	YK → ABS	POST	云卡将放款结果转发给艾博生，用于触发兜底代扣等。
API-11	发起兜底代扣	ABS → 宝付	POST	对于路径B的用户，艾博生在收到放款成功通知后，发起二次代扣。
API-12	还款状态同步	KJ → YK	POST	科技平台将用户的还款状态（如逾期、结清）同步给云卡。
API-13	转发还款状态	YK → ABS	POST	云卡将还款状态进一步转发给艾博生。

七、数据库设计

7.1. 总体设计原则

三方系统数据库物理隔离，互不影响，这是分而治之的核心思想。科技平台作为核心系统，其内部数据库根据业务领域进行逻辑分库（如用户库、订单库、账务库）。

对于读多写少的场景（如商品配置、用户信息查询），可配置MySQL主从复制和读写分离，将读请求路由到从库，提升查询性能，减轻主库压力。

为避免复杂的跨库JOIN查询，允许在不同业务表中适当冗余少量关键字段（如在订单表中冗余用户姓名），通过异步消息保证最终一致性。这是在查询性能和数据一致性之间做出的务实权衡。

7.2. 分库分表策略（科技平台）

考虑到科技平台未来用户量和交易量可能快速增长，对核心业务数据采用分库分表策略。

分片键统一选择user_id。因为大部分核心查询都与用户相关，按用户ID分片可以有效将同一用户的所有数据路由到同一个库/表中，避免跨分片查询。

分片方案采用user_id哈希取模的方式进行水平拆分。初期规划4个库（db_user_0 ~ db_user_3），每个库内核心大表（如借款订单表、还款计划表）再拆分为16张表（kj_loan_order_00 ~ kj_loan_order_15），总分片数为4 x 16 = 64个分片。

分片中间件推荐使用ShardingSphere-JDBC，它以jar包形式提供服务，对应用代码侵入性低，配置相对简单，性能优秀。

7.3. 核心表关系ER图

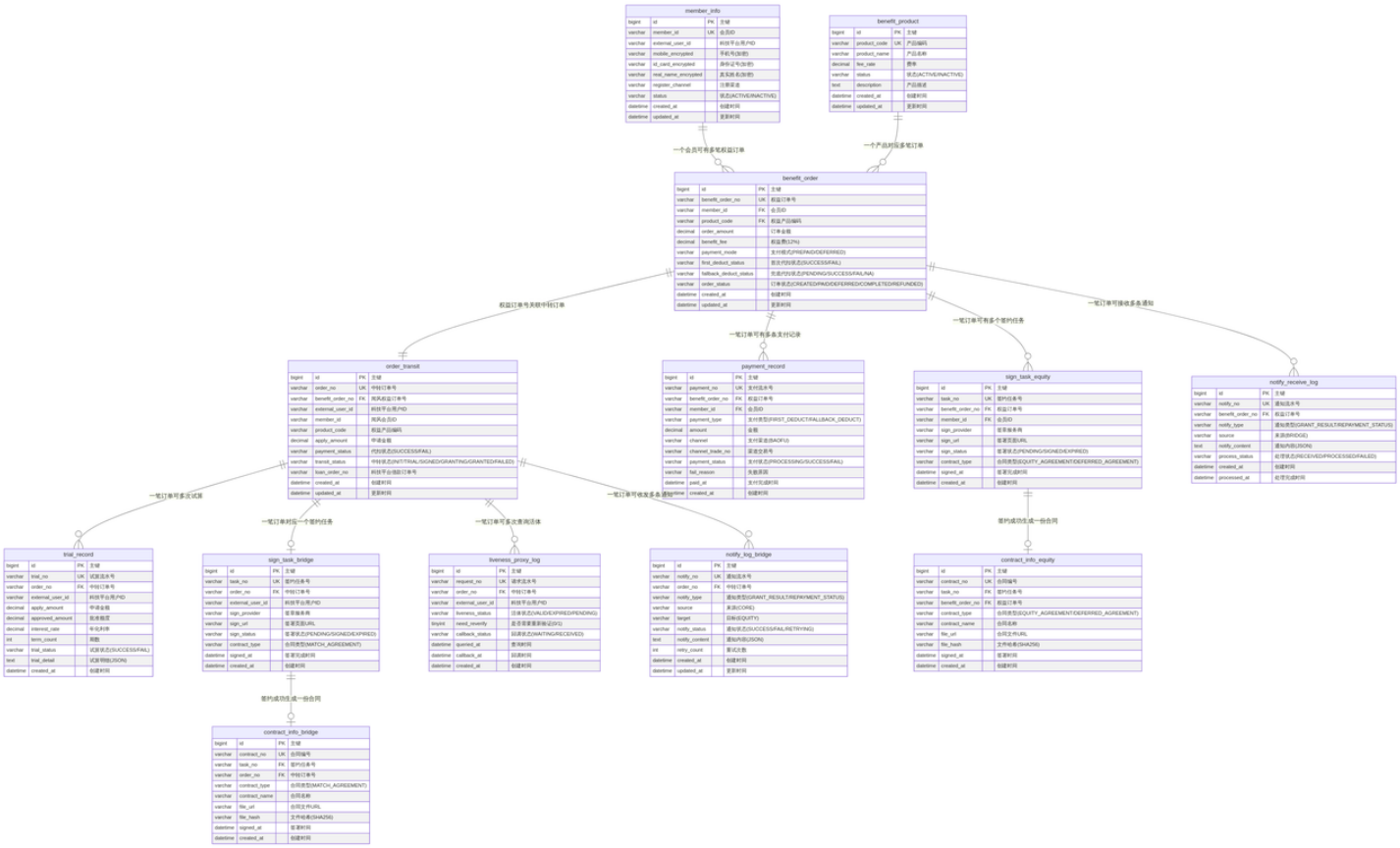
下图展示了云卡、艾博生系统的核心实体及其关系。

NexusFin 数据库ER关系说明

```
nexusfin_equity（闻风）                          nexusfin_bridge（云卡）

┌──────────────────────────────────────┐   ┌──────────────────────────────────────┐
│                                      │   │                                      │
│  member_info ──1:N── benefit_order├──1:1──│order_transit                         │
│                          │           │   │  │                                   │
│                benefit_product       │   │  ├──1:N── trial_record               │
│                          │           │   │  ├──1:1── sign_task                  │
│              ├──1:N─payment_record   │   │  ||            └─1:1─ contract_info  │
│              ├──1:N── sign_task    |  │   │  ├──1:N── liveness_proxy_log         │
│              │    └──1:1── contract_info │  └──1:N── notify_log                 │
│              └──1:N── notify_receive_log │                                      │
│                                      │   │                                      │
└──────────────────────────────────────┘   └──────────────────────────────────────┘

                    │                                          │
                    └──────── benefit_order_no（逻辑纽带）──────────┘
```



八、非功能性设计

8.1. 流量分析与容量规划

假设初期每日导流10,000用户，转化率为5%，则每日产生约500笔借款交易。

核心交易时间段的峰值QPS预计在50左右。考虑到每笔交易涉及多次接口调用（试算、签约、放款等），系统总QPS峰值可能达到500。

在容量规划方面，根据上述技术栈选型，单个Spring Boot节点（4C8G）预计可承载200-300 QPS。为应对峰值流量并预留冗余，科技平台至少需要2个以上节点，云卡和艾博生各需2个节点。

8.2. 资损防控设计

资金安全是本项目的生命线，必须建立事前、事中、事后全链路的资损防控体系。

资损防控体系

事前防控包含三个关键措施。首先是请求合法性校验，对所有写操作请求的参数进行严格校验，防止非法金额、超长逾期等异常数据注入。其次是幂等性保证，所有支付、放款、退款接口必须实现幂等，防止因网络重试导致重复交易。最后是分布式锁，在放款、代扣等关键操作的入口处使用Redisson分布式锁，确保同一时刻只有一个线程能处理同一笔订单，从根本上杜绝并发问题。

事中监控侧重于实时对账和状态机校验。在放款、支付成功后，立即进行内部核心账务表的金额一致性检查，出现差异立即熔断并报警。所有订单的流转必须遵循预设的状态机模型，任何非法的状态跳转（如从"已放款"跳转到"待签约"）都应被拒绝并记录为高危事件。

事后审计通过T+1日终对账实现。每日定时任务执行三方（科技平台、云卡、艾博生）订单的交叉对账，以及与支付渠道（宝付）的账单对账，确保笔数和金额完全一致。对账系统发现任何不平账目，立即触发高级别报警，并提供全链路的request_id和order_no，便于快速定位和追溯问题。

8.3. 安全设计

数据安全方面，全链路强制使用HTTPS/mTLS，确保数据在传输过程中不被窃听和篡改。用户身份证、手机号、银行卡号等敏感信息，采用AES-256对称加密算法加密后存储，密钥通过KMS（密钥管理服务）进行安全管理，实现密钥与数据的物理分离。

应用安全方面，使用MyBatis等ORM框架防止SQL注入，禁止SQL拼接。前端页面对用户输入进行严格转义防止XSS攻击，后端对非GET请求进行CSRF Token校验。定期扫描项目依赖的第三方库，及时修复高危漏洞。

访问控制方面，数据库、缓存等核心组件部署在内网，不允许公网直接访问。应用服务器、数据库账号的权限严格控制在最小必需范围，遵循最小权限原则。

8.4. 缓存策略

暂无

8.5. 限流熔断

暂无

8.6. 监控报警

监控体系覆盖三个维度：系统层（CPU、内存、磁盘、网络IO）、应用层（JVM堆内存、GC、线程池、连接池、QPS、RT）和业务层（核心接口调用量、成功率、失败率；订单量、放款额、代扣成功率等核心业务指标）。

技术方案采用Prometheus采集指标，Grafana进行可视化展示，Alertmanager负责报警规则和通知。报警渠道根据报警级别，通过钉钉、企业微信、短信、电话等多种渠道进行通知，确保问题能被及时响应。

8.7. 日志审计

所有应用输出的日志必须为结构化的JSON格式，并包含trace_id以串联全链路。通过Filebeat采集各服务器上的应用日志，发送至Logstash进行清洗和格式化，最终存入Elasticsearch集群。研发和运维人员通过Kibana进行日志的实时检索、聚合分析和可视化，快速定位线上问题。

对于涉及资金操作的关键日志（如放款、代扣、退款），需要单独存储在审计日志表中，保留至少3年，以满足合规审计要求。审计日志不可删除、不可篡改，需要独立的存储和访问权限控制。

九、Ai工具使用及框架搭建

1）**cursor**；

2）**Claude code team agent 能力（claude --dangerously-skip-permissions --teammate-mode tmux）**

安装：https://cloud.tencent.com/developer/article/2631985

claude-Code: mcp　　1）playwright

skill：1) superpowers；2）planning-with-files

3）**SpecKit（新项目）**https://github.com/github/spec-kit

十、项目名称和代码

NexusFin：Nexus（纽带/连接）+ Finance（金融）

归属	代码库名	业务定位说明
云卡	`nexusfin-bridge`	中转撮合服务（订单中转、试算页、撮合签约、活体代理、消息转发）
云卡	`nexusfin-bridge-h5`	云卡前端工程（试算页、签约页）
艾博生	`nexusfin-equity`	权益分发服务（用户注册、权益订单、电子签章、支付代扣）

艾博生	`nexusfin-equity-h5`	艾博生前端工程（权益购买页、签约页）

数据库名	归属方	说明
`nexusfin_bridge`	云卡	存放订单中转、试算记录、撮合合同等所有中转撮合数据
`nexusfin_equity`	艾博生	存放会员、权益订单、支付流水、签约合同等所有权益分发数据